

RepoMind

Architecture, Design Decisions & Semantic Chunking Implementation Report

Project focus: Codebase Retrieval-Augmented Generation (RAG)

Current milestone: Repository ingestion → scanning → language-aware semantic chunking

Status: Chunking v1 completed; next stage is embeddings + ChromaDB

Executive summary

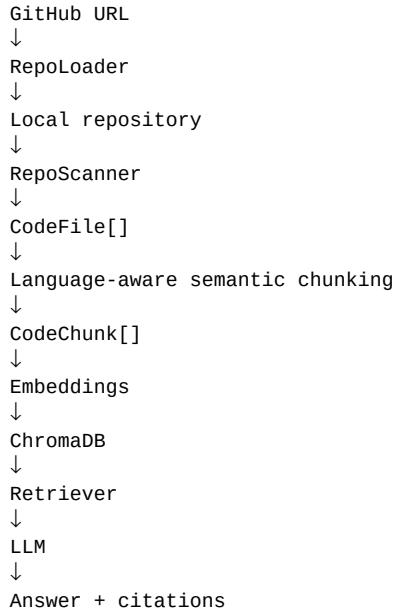
RepoMind is being developed as a codebase RAG system. Its purpose is to ingest a GitHub repository, understand its structure, convert meaningful code and configuration units into retrieval-friendly chunks, embed those chunks, store them in a vector database, retrieve relevant context, and finally provide that context to an LLM for answering questions about the repository. The project deliberately prioritizes semantic structure over naive fixed-size text splitting.

The current implementation has completed repository ingestion, repository scanning, and the first version of language-aware semantic chunking. A real Express repository scan found 165 code files and produced 3,279 chunks in the latest full run. JSON, HTML, and CSS are handled through dedicated structural chunkers, while programming languages use Tree-sitter grammars.

1. Project Goal and RAG Framing

The central problem is not simply storing source files and searching their text. A codebase contains functions, classes, methods, configuration sections, markup structures, styles, dependencies, and relationships between files. RepoMind therefore treats a repository as structured technical knowledge.

Target architecture



The current work ends immediately before embeddings. This boundary is intentional: chunking should first produce stable, meaningful retrieval units before vector storage and retrieval are introduced.

2. Major Design Principles

- **Semantic over arbitrary splitting:** Prefer functions, classes, methods, configuration sections, HTML semantic regions, and CSS rules over fixed character or token windows.
- **Language awareness:** Different programming languages have different AST structures, so one universal node rule is insufficient.
- **Preserve source text:** Chunks retain the original source content rather than reconstructing code from AST nodes.
- **Preserve provenance:** Every chunk records file path and source line boundaries so later retrieval can cite where information came from.
- **Safe fallback:** If a supported file contains no recognized semantic boundary, keep the whole file as one fallback chunk rather than silently losing it.
- **Do not over-engineer early:** YAML, Markdown, relationship graphs, reranking, and other refinements are deferred until retrieval experiments demonstrate a need.
- **Functional architecture:** The implementation is module/function oriented rather than class-heavy.

3. Repository Ingestion

The repository loader uses **simple-git** to clone a GitHub repository into a local data directory. The repository name is extracted from the URL, and an existing local repository is reused instead of being cloned again.

```
data/  
■■■ repos/
```

```
■■■ <repoName>/
```

The loader returns a Repository object containing the repository name, original URL, and local path. This keeps ingestion separate from scanning and chunking.

4. Repository Scanning

The scanner discovers files using **fast-glob**, applies **.gitignore** rules with the **ignore** package, explicitly excludes **.git** and **node_modules**, determines language from file extensions, skips unsupported extensions, and rejects files larger than 1 MB.

Important separation of responsibilities: fast-glob discovers files; chunkers understand file contents.

The scanner produces an in-memory CodeFile representation:

```
interface CodeFile {  
  path: string;  
  language: string;  
  content: string;  
  size: number;  
}
```

5. The CodeChunk Data Model

A chunk is the fundamental unit passed to the future embedding stage. The current model deliberately contains both content and structural metadata.

```
interface CodeChunk {  
  content: string;  
  filePath: string;  
  language: string;  
  startLine: number;  
  endLine: number;  
  symbol?: string;  
  type: string;  
}
```

Why this metadata matters: content is what gets embedded, while file path, language, symbol, type, and line range provide provenance, filtering opportunities, debugging information, and future citation support.

6. Why Tree-sitter Was Chosen

Tree-sitter provides syntax trees for programming languages. Instead of treating source code as a plain string, RepoMind can identify syntactic constructs such as function declarations, class declarations, method declarations, interfaces, enums, and type declarations.

```
CodeFile
↓
Tree-sitter parser
↓
Syntax tree
↓
Meaningful semantic nodes
↓
CodeChunk[]
```

The syntax tree is an in-memory representation. It is not stored as a separate artifact. The implementation extracts the relevant nodes and slices the original source using the nodes' source positions.

Compatible grammar versions were pinned where necessary so the installed language grammars remain compatible with the selected Tree-sitter core version. This avoided forcing incompatible native dependency trees.

7. Programming Language Semantic Chunking

The programming-language chunker parses a file with the appropriate grammar, walks the syntax tree, selects language-specific semantic node types, extracts the original source text for each selected node, and builds CodeChunk metadata.

Currently implemented language grammars

Language	Semantic boundaries currently recognized
JavaScript	functions, function nodes, classes, methods, arrow functions
TypeScript	functions, classes, methods, arrow functions, interfaces, type aliases, enums
Python	functions, classes
Java	classes, methods, constructors, interfaces, enums
C	functions, structs
C++	classes, functions, structs
Go	functions, methods, type declarations

8. Language-Aware Symbol Extraction

A major implementation issue was that Tree-sitter does not expose names identically across languages. The implementation therefore uses a generic *name* field where available and language-specific logic where required.

For C and C++, function names may be nested inside declarators, so a recursive identifier search is used. For Go type declarations, the implementation navigates through *type_spec* to find the *type_identifier*. This avoids producing chunks with missing symbols when the grammar structure differs.

Example result:

```
JAVA
class UserService
method createUser
method deleteUser

C
```

```
function add
function main

GO
type User
function add
function main
```

9. File-Level Fallback Strategy

A supported file can contain code that does not match the current semantic-node rules. Returning zero chunks would mean that content effectively disappears from the RAG corpus. RepoMind therefore creates one file-level fallback chunk when no semantic nodes are found.

```
{
  content: file.content,
  filePath: file.path,
  language: file.language,
  startLine: 1,
  endLine: totalLines,
  symbol: undefined,
  type: "file"
}
```

This is considered a safety mechanism rather than the ideal chunking strategy. Semantic chunks remain preferable because they provide more precise retrieval units.

10. HTML Chunking

HTML is not treated like a programming language. The chunker uses Tree-sitter HTML to identify elements, but it does not create a chunk for every small nested element. Instead, it recognizes semantically meaningful boundary tags.

Semantic HTML boundaries include: html, head, body, header, nav, main, section, article, aside, footer, form, table, script, and style.

Small elements remain inside their surrounding semantic region. This prevents a page from becoming hundreds of tiny retrieval fragments while retaining meaningful page structure.

11. CSS Chunking

CSS is similarly handled with its own structural rules. The current chunker recognizes regular rule sets and media statements.

```
.navbar { ... }  
→ rule chunk  
  
.login-form { ... }  
→ rule chunk  
  
@media (...) {  
  ...  
}  
→ media chunk
```

A media query remains a coherent unit, including its nested rules. This is more useful for retrieval than splitting each nested declaration independently.

12. JSON Chunking

JSON was deliberately added because repositories commonly contain package.json and other structured configuration files. JSON is not passed through the programming-language grammar map. It has a dedicated document chunker.

For a JSON object, top-level **pair** nodes are used as semantic boundaries. Nested entries such as individual dependencies are kept together rather than creating one embedding per dependency.

Actual Express package.json result: 15 chunks.

```
dependencies → lines 34-63  
devDependencies → lines 64-81  
engines → lines 82-84  
files → lines 85-90  
scripts → lines 91-98
```

This demonstrates the intended behavior: large structured configuration sections become coherent retrieval units. The current implementation also produces small chunks for scalar metadata such as name, version, description, license, and repository. Grouping those small metadata fields is a possible future refinement, but it is not required before moving to embeddings.

13. Dispatcher Architecture

A central dispatcher keeps format-specific behavior isolated. It routes HTML, CSS/SCSS, and JSON to dedicated chunkers and sends the supported programming languages through the Tree-sitter code chunker.

```
switch (file.language) {  
  case "html":  
    return chunkHTML(file);
```

```
case "css":  
case "scss":  
return chunkCSS(file);  
  
case "json":  
return chunkDocument(file);  
  
default:  
return chunkCodeFile(file);  
}
```

14. Full Express Repository Validation

The implementation was tested against a real Express repository rather than only isolated toy files. The scanner found 165 code files in the repository.

```
Scanning repository...
Found 165 code files
```

The final full run reported 3,279 total chunks. Representative JavaScript files produced substantial numbers of semantic chunks, including `application.js` (27), `request.js` (21), `response.js` (50), and `test/app.router.js` (219).

The run also confirmed that `package.json` is processed by the JSON chunker and produces 15 chunks. HTML and CSS files in the repository are also routed through their dedicated chunkers.

Observed unsupported formats:

- YAML files such as `.eslintrc.yml` and GitHub workflow files currently report Unsupported language: `yaml`.
- Markdown files such as `Readme.md` and `History.md` currently report Unsupported language: `markdown`.
- These are known scope limitations, not failures of the implemented programming-language, HTML, CSS, or JSON chunkers.

15. Important Implementation Decisions

Decision	Reason
Semantic AST chunking	Preserves meaningful code boundaries for retrieval.
Language-specific node maps	AST structures differ between languages.
Original source slicing	Preserves exact source formatting/content.
Line metadata	Enables provenance and future citations.
File fallback	Prevents supported content from disappearing.
Dedicated HTML/CSS chunkers	Markup and styling have different semantic structures.
Dedicated JSON chunker	Configuration sections are useful RAG units.
Defer YAML/Markdown	Avoid unnecessary complexity before retrieval validation.
No relationship graph yet	Relationships can be added after baseline retrieval works.
No embedding yet	Chunk boundaries should be stabilized before vectorization.

16. Current Limitations and Deferred Work

The current milestone is intentionally incomplete in several areas. These are recorded so that future work does not accidentally treat Chunking v1 as a complete code-understanding system.

- YAML is not currently chunked. An attempted Tree-sitter YAML installation required native Windows build tooling, so YAML was deferred rather than installing a compiler toolchain solely for this feature.
- Markdown is not currently chunked.
- C++ functions inside classes can currently be labeled as function; a future parent-context refinement can classify such nodes as methods.
- JavaScript/TypeScript arrow-function symbol extraction can be improved by inspecting the surrounding variable declarator.

- Small JSON metadata fields could be grouped to reduce very small embeddings.
- Cross-file relationships such as controller → service → repository are not yet explicitly represented.
- No hybrid search or reranking has been implemented yet.
- Retrieval quality has not yet been empirically evaluated because embeddings and ChromaDB are the next stage.

17. Why Chunking Is Considered Complete for This Milestone

The objective of this phase is not to support every possible file format or anticipate every future retrieval problem. The objective is to create a strong semantic baseline that can be embedded and evaluated.

The implementation now covers the principal programming languages selected for the project plus HTML, CSS, and JSON. A real repository has been processed successfully, producing thousands of semantic or fallback chunks. The remaining limitations are explicit and isolated.

At this point, continuing to add chunking rules without testing retrieval would risk optimizing the ingestion layer based on assumptions. The next step should instead be to embed the current chunks, store them in ChromaDB, run real queries, inspect failures, and only then modify chunking when evidence indicates that chunk boundaries are responsible.

18. Next Phase: Embeddings + ChromaDB

```

CodeChunk[]
↓
Embedding model
↓
Vector embeddings
↓
ChromaDB
↓
Similarity search
↓
Top-K relevant chunks
↓
Context assembly
↓
LLM
↓
Answer + source citations

```

The next engineering goal is therefore not to make chunking infinitely sophisticated. It is to determine whether the current semantic units produce useful retrieval results. Retrieval evaluation will provide concrete evidence for future refinements.

19. Current Project Status

Component	Status
GitHub repository loading	Completed
Local repository management	Completed
Repository scanning	Completed
Language detection	Completed for selected extensions
JavaScript / TypeScript chunking	Completed v1
Python chunking	Completed v1
Java chunking	Completed v1

Component	Status
C / C++ chunking	Completed v1
Go chunking	Completed v1
HTML chunking	Completed v1
CSS / SCSS chunking	Completed v1
JSON chunking	Completed v1
YAML chunking	Deferred
Markdown chunking	Deferred
Embeddings	Next
ChromaDB	Next
Retriever	Future
LLM answer generation	Future

Conclusion

RepoMind has completed its first substantial RAG foundation: repository ingestion, file discovery, language classification, and semantic chunk extraction. The architecture intentionally keeps these concerns modular so that embeddings, vector storage, retrieval, and later relationship-aware context can be added without rewriting the ingestion layer. Chunking v1 is therefore frozen as the baseline for the next phase.

Source basis: This report documents the implementation decisions, tests, and results established during the RepoMind development work. The latest repository test output reports 165 files and 3,279 total chunks; package.json produces 15 JSON chunks, while YAML and Markdown remain unsupported in the current implementation.